```python
import os
import pandas as pd
import numpy as np
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import classification_report, accuracy_score

# Define file paths
model_path = "water_model_large.pkl"
scaler_path = "scaler_large.pkl"
encoders_path = "encoders_large.pkl"
columns_path = "columns_large.pkl"

# □ Part 1: Train the model only if not already trained
if not os.path.exists(model_path):
    print("□ Training model from scratch...")

    # Load your dataset
    df = pd.read_csv("large_dataset.csv")

    # Select features (added Pipeline_Age and Pipeline_Material)
    features = ['pH', 'Nitrate', 'Turbidity', 'Chloride', 'Color',
                'Source', 'City', 'Pipeline_Age', 'Pipeline_Material']
    target = 'Target'
    df = df[features + [target]]

    # Handle missing values
    for col in df.columns:
        if df[col].dtype == 'object':
            df[col].fillna(df[col].mode()[0], inplace=True)
        else:
            df[col].fillna(df[col].median(), inplace=True)

    # Encode categorical variables
    label_encoders = {}
    for col in ['Color', 'Source', 'City', 'Pipeline_Material']:
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])
        label_encoders[col] = le

    # Separate features and target
    X = df.drop(target, axis=1)
    y = df[target]

    # Fix label inversion (original 0 = drinkable)
    y = y.map({0: 1, 1: 0})

    # Scale numerical features
```

```
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    # Split into training and test sets
    X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.2, random_state=42)

    # Train the model
    model = HistGradientBoostingClassifier(class_weight='balanced',
random_state=42)
    model.fit(X_train, y_train)

    # Evaluate model
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"□ Model trained successfully! Test Accuracy: {acc:.4f}")
    print(classification_report(y_test, y_pred))

    # Save model and transformers
    joblib.dump(model, model_path)
    joblib.dump(scaler, scaler_path)
    joblib.dump(label_encoders, encoders_path)
    joblib.dump(X.columns.tolist(), columns_path)

else:
    print("□ Pre-trained model found and loaded.")
```

```
□ Training model from scratch...

/var/folders/0q/dthm7v6x27v8rytq6ytypzm00000gn/T/
ipykernel_28650/1370017460.py:34: FutureWarning: A value is trying to
be set on a copy of a DataFrame or Series through chained assignment
using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df[col].fillna(df[col].median(), inplace=True)
/var/folders/0q/dthm7v6x27v8rytq6ytypzm00000gn/T/ipykernel_28650/13700
17460.py:32: FutureWarning: A value is trying to be set on a copy of a
DataFrame or Series through chained assignment using an inplace
method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
```

```
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df[col].fillna(df[col].mode()[0], inplace=True)

 Model trained successfully! Test Accuracy: 0.8435
              precision    recall  f1-score   support

           0       0.63      0.79      0.70     48105
           1       0.93      0.86      0.89    161610

    accuracy                           0.84    209715
   macro avg       0.78      0.82      0.80    209715
weighted avg       0.86      0.84      0.85    209715


# □ Part 2: Load trained components
model = joblib.load(model_path)
scaler = joblib.load(scaler_path)
label_encoders = joblib.load(encoders_path)
feature_columns = joblib.load(columns_path)

# □ Part 3: Prediction function (updated for new features)
def predict_drinkability(pH, Nitrate, Turbidity, Chloride, Color,
Source, City, Pipeline_Age, Pipeline_Material):
    data = pd.DataFrame([{
        'pH': pH,
        'Nitrate': Nitrate,
        'Turbidity': Turbidity,
        'Chloride': Chloride,
        'Color': Color,
        'Source': Source,
        'City': City,
        'Pipeline_Age': Pipeline_Age,
        'Pipeline_Material': Pipeline_Material
    }])

    # Encode categorical columns
    for col, le in label_encoders.items():
        data[col] = le.transform(data[col])

    # Arrange column order
    data = data[feature_columns]

    # Scale numeric columns
    data_scaled = scaler.transform(data)
```

```
    # Predict
    pred = model.predict(data_scaled)[0]
    return "Drinkable □" if pred == 1 else "Not Drinkable □"

# □ Part 4: Sample Predictions
print(predict_drinkability(7.2, 10.0, 0.4, 110.0, 'Colorless',
'River', 'Fremont', 15, 'PVC'))          # □
print(predict_drinkability(5.5, 60.0, 3.2, 280.0, 'Light Yellow',
'Ground', 'San Francisco', 45, 'Steel')) # □
print(predict_drinkability(6.8, 8.0, 0.5, 95.0, 'Near Colorless',
'Ground', 'Alameda', 10, 'PVC'))       # □
print(predict_drinkability(4.9, 48.0, 5.5, 330.0, 'Faint Yellow',
'River', 'Oakland', 40, 'Cast Iron'))   # □
```

```
Not Drinkable □
```